CENTRAL INSTITUTE OF TECHNOLOGY KOKRAJHAR
(Deemed to be University under MoE, Govt. of India)

Software Engineering Project

UCSE672

Group A

Topic: Flight Management System

Group Members:

Dharam Dev Basumatary (202302021032)

Manosh Priya Gogoi (202302021038)

David Basumatary (202302021046)

SOFTWARE REQUIREMENTS SPECIFICATION

Flight Ticket Management System

1. Introduction

1.1. Purpose

The purpose of this document is to describe the functional and non-functional requirements of the Flight Ticket Booking and Management System. This system is designed to allow users to search, book, and manage flight tickets online. It also provides administrative functionalities for managing flight details, cancellations, and bookings.

1.2. Scope

The Flight Ticket Booking and Management System is a web-based application that enables customers to register, login, search flights, book tickets, make payments, and request cancellations. The system also allows administrators to manage flights, modify flight details, cancel flights, approve or reject cancellation requests, and monitor bookings.

The system aims to automate the flight booking process and improve efficiency, transparency, and user convenience.

1.3. Definitions

User – A customer who registers and books flight tickets.

Admin – An authorized administrator who manages flight and booking information.

Booking – A confirmed reservation of a flight by a user.

Cancellation Request – A request initiated by the user to cancel a booked ticket.

1.4. Overview

This document describes the overall system functionality, system features, user requirements, system constraints, and performance requirements.

2. Overall Description

2.1. Product Perspective

The system is a standalone web-based application developed using HTML and CSS for the frontend, Node.js for the backend, and MySQL as the database management system. The application follows a client-server architecture.

2.2. Product Functions

The system provides the following major functions:

- Landing page for navigation
- Separate login and registration for users and admin
- Flight search based on destination, date, number of passengers, and class
- Ticket booking after successful payment
- My Flights section to view bookings
- Ticket cancellation request feature
- Admin dashboard for managing flights and bookings
- Admin approval or rejection of cancellation requests
- Automatic restriction of cancellation within 12 hours before departure

2.3. User Classes

User (Customer):

Users can register, login, search flights, book tickets, make payments, view bookings, and request cancellations.

Admin:

Admin can login, add new flights, edit flight details, change price, update capacity, change destination, remove flights, cancel flights with a reason, and approve or reject ticket cancellation requests.

2.4. Operating Environment

- Web browser (Chrome, Firefox, Edge)
- Server running Node.js
- MySQL database server
- Minimum 4GB RAM

2.5. Constraints

- The system uses MySQL as the database.
- The system must enforce a 12-hour rule for ticket cancellation.
- The system must restrict unauthorized access to admin functionalities.

3. Functional Requirements

3.1. Landing Page

FR1: The system shall provide a landing page with options for User Login and Admin Login.

FR2: The system shall provide navigation to registration pages.

3.2. User Registration and Login

FR3: The system shall allow users to register using name, email, password, and phone number.
FR4: The system shall validate login credentials before granting access.

3.3. Admin Login

FR5: The system shall allow admin to login using authorized credentials.
FR6: The system shall redirect admin to an admin dashboard after successful login.

3.4. Flight Search

FR7: The system shall allow users to search flights using destination, date, number of passengers, and travel class.
FR8: The system shall display available flights that match the search criteria.

3.5. Ticket Booking

FR9: The system shall allow users to select a flight and proceed to booking.
FR10: The system shall calculate the total price based on number of passengers and selected class.
FR11: The system shall redirect the user to the payment page.

3.6. Payment Processing

FR12: The system shall confirm booking only after successful payment.
FR13: The system shall store booking details in the database after payment confirmation.

3.7. My Flights Section

FR14: The system shall provide a “My Flights” section for users.
FR15: The system shall display flight details, booking status, and travel information.
FR16: The system shall allow users to request cancellation of tickets.

3.8. Cancellation Management

FR17: The system shall allow ticket cancellation requests only if the request is made at least 12 hours before flight departure.
FR18: If the request is made within 12 hours of departure, the system shall automatically reject the cancellation request.
FR19: When a valid cancellation request is submitted, the system shall update booking status to “Cancellation Requested”.
FR20: The system shall notify the admin about pending cancellation requests.
FR21: The admin shall be able to approve or reject cancellation requests.
FR22: If approved, the booking status shall be updated to “Cancelled” and

available seats shall be updated.
FR23: If rejected, the booking status shall be updated to “Cancellation Rejected”.

3.9. Flight Management (Admin)

FR24: The admin shall be able to add new flights.
FR25: The admin shall be able to edit flight details including flight name, destination, departure time, arrival time, capacity, and price.
FR26: The admin shall be able to change flight price and capacity.
FR27: The admin shall be able to cancel a flight and provide a cancellation message.
FR28: The system shall update all related bookings if a flight is cancelled.
FR29: The admin shall be able to remove flights from the system.

4. Non-Functional Requirements

4.1. Performance

The system shall display search results within 3 seconds under normal operating conditions.

4.2. Security

Passwords shall be securely stored in the database.
Only authenticated users shall access user features.
Only authorized admin shall access administrative features.

4.3. Reliability

The system shall maintain accurate booking and payment records.
The system shall prevent duplicate bookings for the same seat.

4.4. Usability

The system interface shall be simple, clear, and easy to navigate.

4.5. Availability

The system shall be accessible 24 hours a day.

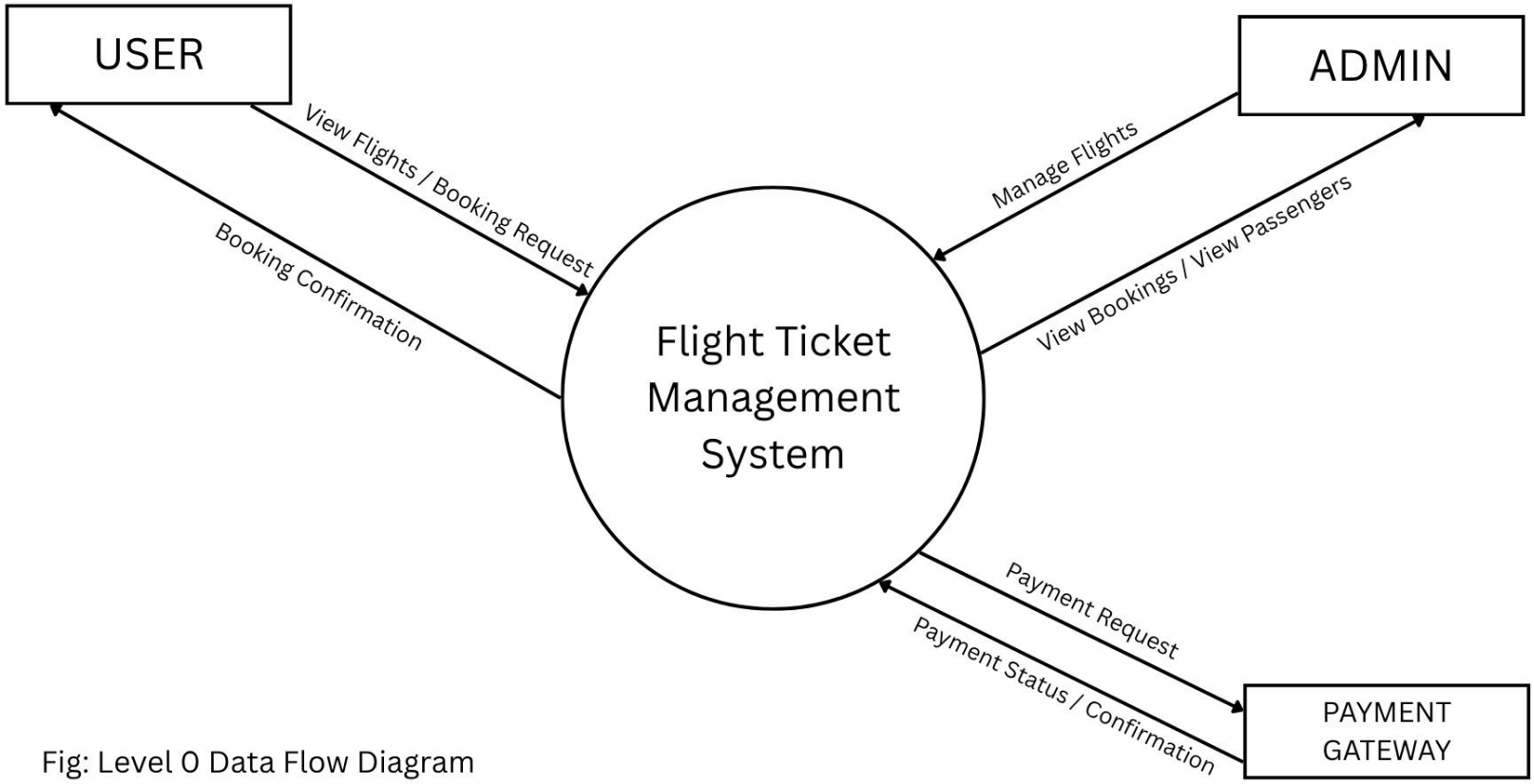

Fig: Level 0 Data Flow Diagram
(Context Diagram)

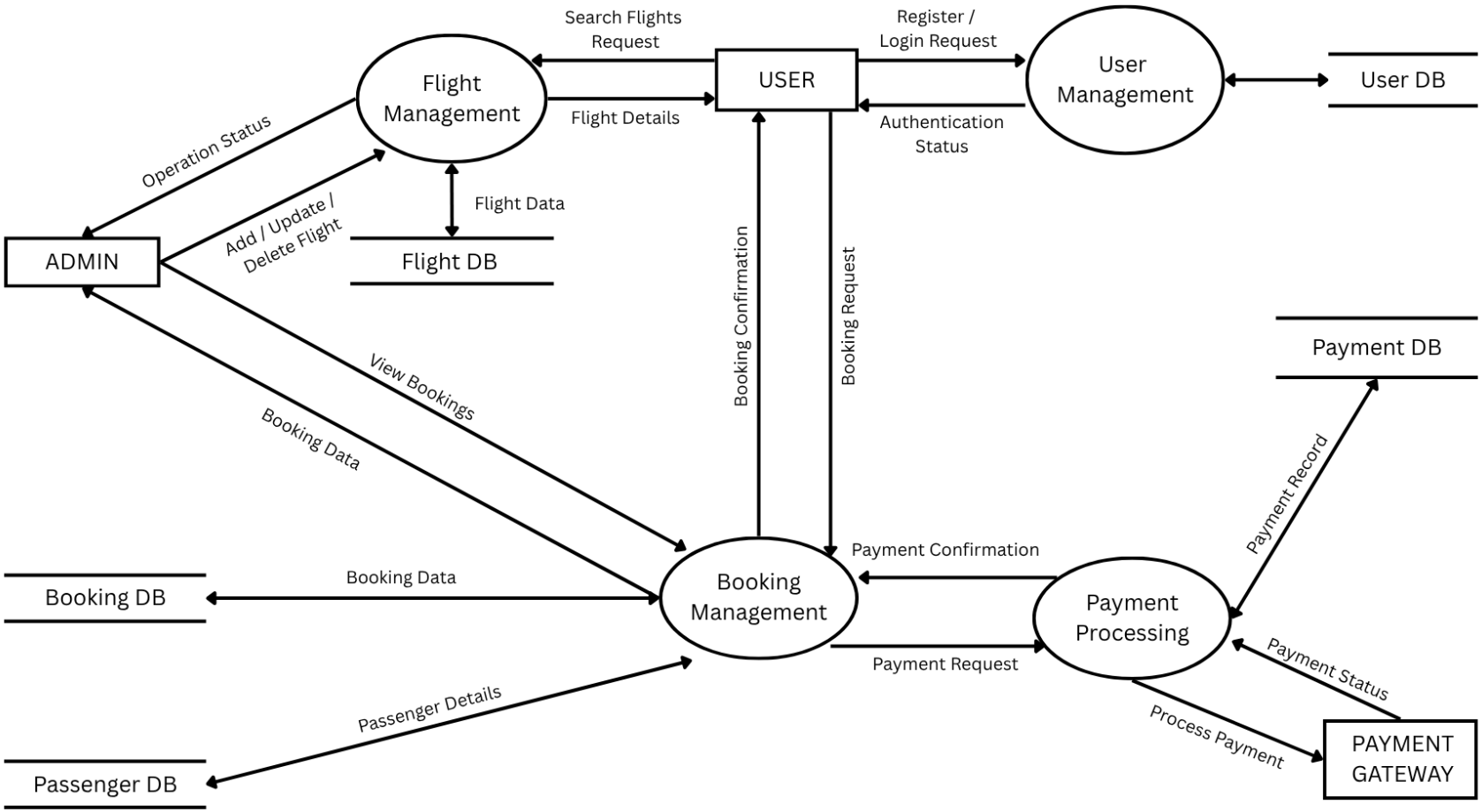

Fig: Level 1 Data Flow Diagram

DATABASE DESIGN

```
mysql> use flight_booking_system;
Database changed
mysql> show tables;
+-----------------------------------+
| Tables_in_flight_booking_system |
+-----------------------------------+
| bookings                          |
| flights                           |
| passengers                        |
| payments                          |
| users                             |
+-----------------------------------+
```

1. **Users Table**
   Stores both customers and admin (using role column).

   Users(
   user_id PK,
   name,
   email,
   password,
   phone,
   role
)

```
mysql> desc users;
+----------+-------------+------+-----+---------+----------------+
| Field    | Type        | Null | Key | Default | Extra          |
+----------+-------------+------+-----+---------+----------------+
| user_id  | int         | NO   | PRI | NULL    | auto_increment |
| name     | varchar(100) | YES  |     | NULL    |                |
| email    | varchar(100) | YES  | UNI | NULL    |                |
| password | varchar(255) | YES  |     | NULL    |                |
| phone    | varchar(15)  | YES  |     | NULL    |                |
| role     | varchar(20)  | YES  |     | NULL    |                |
+----------+-------------+------+-----+---------+----------------+
```

   Purpose: Stores login and identity information.

2. **Flights Table**
   Stores flight information managed by admin.

   Flights(
   flight_id PK,
   flight_number,
   source,
   destination,
   departure_time,
   arrival_time,
   capacity,
   available_seats,
   price,
   status,
   cancellation_message
)

```
mysql> desc flights;
+----------------------+---------------+------+-----+---------+----------------+
| Field                | Type          | Null | Key | Default | Extra          |
+----------------------+---------------+------+-----+---------+----------------+
| flight_id            | int           | NO   | PRI | NULL    | auto_increment |
| flight_number        | varchar(20)   | YES  |     | NULL    |                |
| source               | varchar(100)  | YES  |     | NULL    |                |
| destination          | varchar(100)  | YES  |     | NULL    |                |
| departure_time       | datetime      | YES  |     | NULL    |                |
| arrival_time         | datetime      | YES  |     | NULL    |                |
| capacity             | int           | YES  |     | NULL    |                |
| available_seats      | int           | YES  |     | NULL    |                |
| price                | decimal(10,2) | YES  |     | NULL    |                |
| status               | varchar(20)   | YES  |     | NULL    |                |
| cancellation_message | text          | YES  |     | NULL    |                |
+----------------------+---------------+------+-----+---------+----------------+
```

   Purpose: Stores all flight-related details.

3. **Bookings Table**
   Stores booking record created after payment.

   Bookings(
   booking_id PK,
   user_id FK,
   flight_id FK,
   booking_time,
   passengers_count,
   travel_class,

total_amount,
status,
cancellation_reason
)

```
mysql> desc bookings;
+--------------------+---------------+------+-----+-------------------+-------------------+
| Field              | Type          | Null | Key | Default           | Extra             |
+--------------------+---------------+------+-----+-------------------+-------------------+
| booking_id         | int           | NO   | PRI | NULL              | auto_increment    |
| user_id            | int           | YES  | MUL | NULL              |                   |
| flight_id          | int           | YES  | MUL | NULL              |                   |
| booking_time       | datetime      | YES  |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
| passengers_count   | int           | YES  |     | NULL              |                   |
| travel_class       | varchar(20)   | YES  |     | NULL              |                   |
| total_amount       | decimal(10,2) | YES  |     | NULL              |                   |
| status             | varchar(30)   | YES  |     | NULL              |                   |
| cancellation_reason | text          | YES  |     | NULL              |                   |
+--------------------+---------------+------+-----+-------------------+-------------------+
```

Foreign Keys:
user_id → Users(user_id)
flight_id → Flights(flight_id)

Purpose: Connects user and flight.

4. **Passengers Table**
Stores each passenger under a booking.

Passengers(
passenger_id PK,
booking_id FK,
name,
age,
gender,
seat_number
)

```
mysql> desc passengers;
+--------------+--------------+------+-----+---------+----------------+
| Field        | Type         | Null | Key | Default | Extra          |
+--------------+--------------+------+-----+---------+----------------+
| passenger_id | int          | NO   | PRI | NULL    | auto_increment |
| booking_id   | int          | YES  | MUL | NULL    |                |
| name         | varchar(100) | YES  |     | NULL    |                |
| age          | int          | YES  |     | NULL    |                |
| gender       | varchar(10)  | YES  |     | NULL    |                |
| seat_number  | varchar(10)  | YES  |     | NULL    |                |
+--------------+--------------+------+-----+---------+----------------+
```

Foreign Key:
booking_id → Bookings(booking_id)

Purpose: Stores individual passenger details.

5. **Payments Table**
Stores payment details for booking.

Payments(
payment_id PK,
booking_id FK,
payment_date,
amount,
payment_status
)

```
mysql> desc payments;
+----------------+---------------+------+-----+-------------------+-------------------+
| Field          | Type          | Null | Key | Default           | Extra             |
+----------------+---------------+------+-----+-------------------+-------------------+
| payment_id     | int           | NO   | PRI | NULL              | auto_increment    |
| booking_id     | int           | YES  | UNI | NULL              |                   |
| payment_date   | datetime      | YES  |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
| amount         | decimal(10,2) | YES  |     | NULL              |                   |
| payment_status | varchar(20)   | YES  |     | NULL              |                   |
+----------------+---------------+------+-----+-------------------+-------------------+
```

Foreign Key:
booking_id → Bookings(booking_id)
Purpose: Stores payment transaction info.

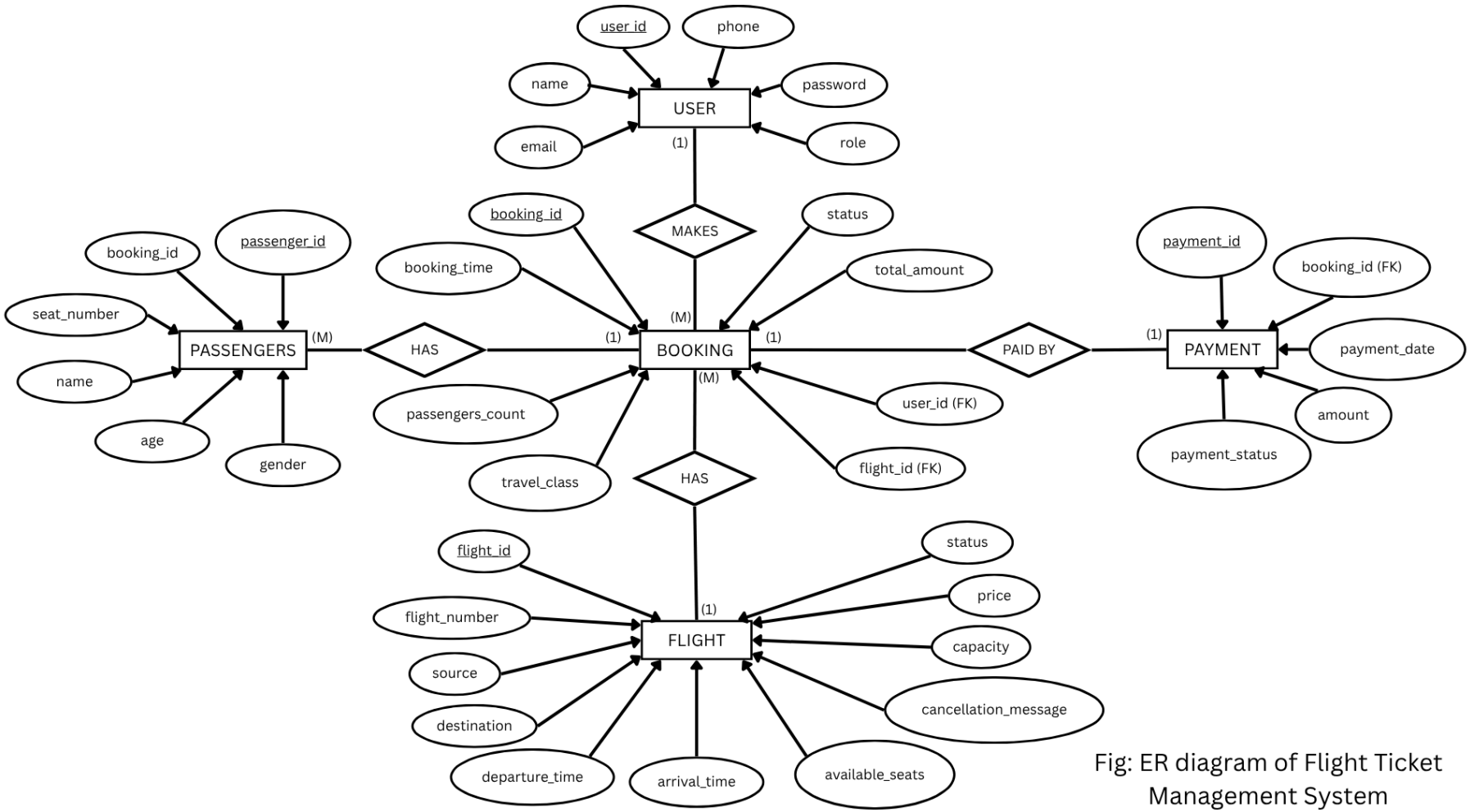

Fig: ER diagram of Flight Ticket Management System